



SINCRONIZACIÓN DE PROCESOS

DEPARTAMENTO DE CIENCIAS E INGENIERÍA
DE LA COMPUTACIÓN

UNIVERSIDAD NACIONAL DEL SUR

Agenda

1. Fundamentos
2. El Problema de la Sección Crítica
3. Solución a la sección crítica para 2 procesos (Algoritmo de Peterson) y para n procesos (Algoritmo del Panadero)
4. Soluciones por Hardware
5. Soluciones por Software
 1. Locks
 2. Semáforos
 3. Monitores
6. Problemas Clásicos
7. Ejemplos de Sincronización en los Sistemas Operativos

Ejemplos de Sincronización

- Windows
- Linux
- Solaris
- Pthreads

Sincronización Windows

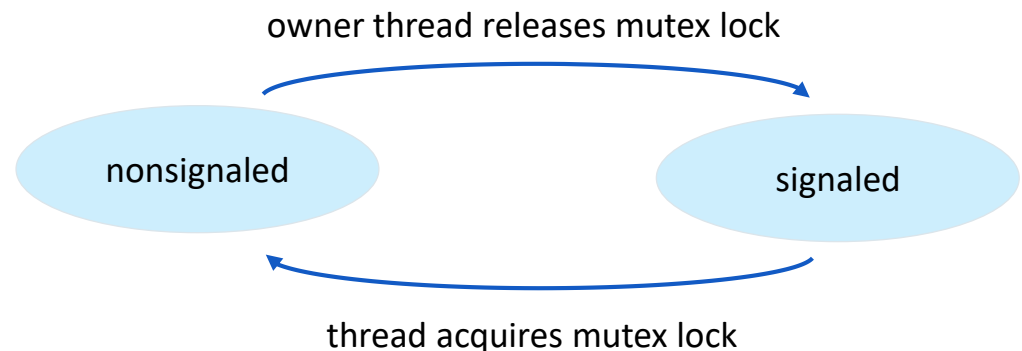
En el kernel

- Usa máscaras de interrupción para proteger el acceso a recursos globales en sistemas mono procesador
- Usa **spinlocks** en sistemas multiprocesador

Sincronización hilos

- También provee **dispatcher objects** los cuales actúan como mutex locks, semáforos, eventos y timers.
- Los *Dispatcher objects* pueden proveer **eventos**
 - Un evento actúa como una variable de condición

Los estados del *MUTEX*
DISPATCHER OBJECT



Sincronización en Linux

- Linux:
 - Antes de la versión 2.6 no era un kernel totalmente apropiativo.
 - Deshabilita las interrupciones para implementar secciones críticas cortas
- Linux provee:
 - mutex-locks
 - semáforos
 - spin locks

Un procesador	Múltiples procesadores
Deshabilitar apropiación kernel	Acquire spin lock
Habilitar apropiación kernel	Release spin lock

Sincronización en SOLARIS

Implementa una variedad de locks para soportar multitasking, multithreading (incluyendo threads en tiempo real), y multiprocesamiento.

- Lock Exclusión Mutua
- Semáforos
- Lock Lectores-Escritor
- Variables de condición

Sincronización en Posix APIs

- APIs Pthreads son independientes de los SOs
- Proveen:
 - Locks mutex
 - Variables de condición
- Extensiones no portables incluyen:
 - Locks lector-escritor
 - spin locks

Sincronización en Posix APIs

Incluir
`pthread.h`

- Operaciones sobre mutex-lock

- `int pthread_mutex_lock (pthread_mutex_t *mutex)`
- `int pthread_mutex_unlock (pthread_mutex_t *mutex)`
- `int pthread_mutex_trylock (pthread_mutex_t *mutex)`

- Operaciones sobre variables de condición

- `int pthread_cond_wait (pthread_cond_t *cond, pthread_mutex_t *mutex)`
- `int pthread_cond_signal (pthread_cond_t *cond)`

Sincronización en Posix APIs

○ Operaciones sobre semáforos

- `int sem_init(sem_t *sem, int pshared, unsigned int value);`
- `int sem_wait(sem_t *sem);`
- `int sem_trywait(sem_t *sem);`
- `int sem_post(sem_t *sem);`



**Incluir
semaphore.h**

Bibliografía:

- Silberschatz, A., Gagne G., y Galvin, P.B.; "Operating System Concepts", 7ma Edición 2009, 9na Edición 2012, 10ma Edición 2018.
- Stallings, W. "Operating Systems: Internals and Design Principles", Prentice Hall, 7ma Edición 2011, 8va Edición 2014, 9na Edición 2018.
- Tanenbaum, A.; "Modern Operating Systems", Addison-Wesley, 3ra. Edición 2008, 4ta. Edición 2014.